

We can easily switch back to another branch (in this case master branch), work separately on it and commit the changes, which is as shown in Figure 2.

```
(base) ekta@git-demo$ git checkout -b feature
Switched to a new branch 'feature'
(base) ekta@git-demo$ echo "branching in git" > branch_file.txt
(base) ekta@git-demo$ git add branch_file.txt
(base) ekta@git-demo$ git commit -m "first commit on a branch"
[feature d84d016] first commit on a branch
1 file changed, 1 insertion(+)
create mode 100644 branch_file.txt
```

Figure 1: Create and check out a new branch

```
(base) ekta@git-demo$ git checkout master
Switched to branch 'master'
(base) ekta@git-demo$ echo "new line in file1" >> file_1.txt
(base) ekta@git-demo$ git add file_1.txt
(base) ekta@git-demo$ git commit -m "master branch commit"
[master 5626b79] master branch commit
1 file changed, 1 insertion(+)
```

Figure 2: Check out the master branch

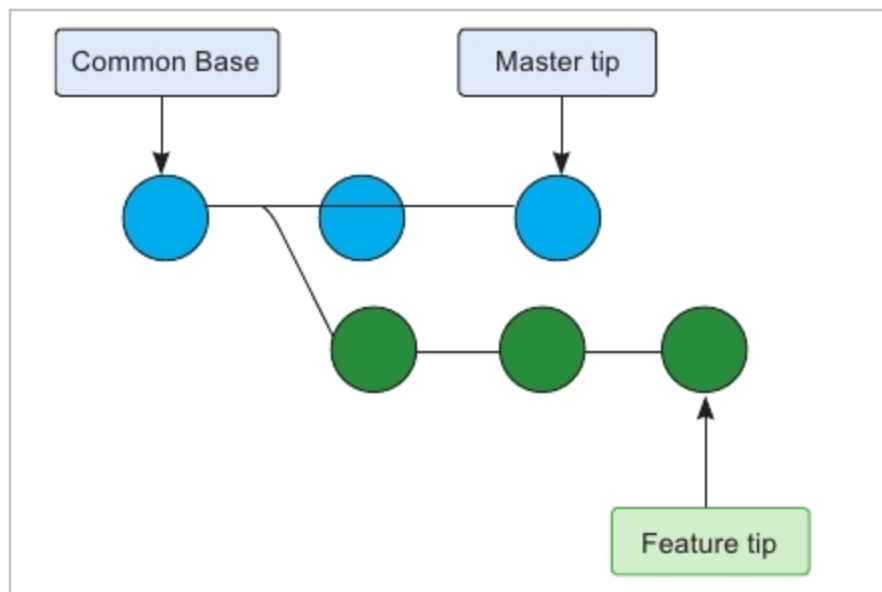


Figure 3: Snapshots of branches before merge

```
(base) ekta@git-demo$ git checkout feature
Switched to branch 'feature'
(base) ekta@git-demo$ git merge master
Merge made by the 'recursive' strategy.
 file_1.txt | 1 +
 1 file changed, 1 insertion(+)
```

Figure 4: Command for merging master branch to feature branch

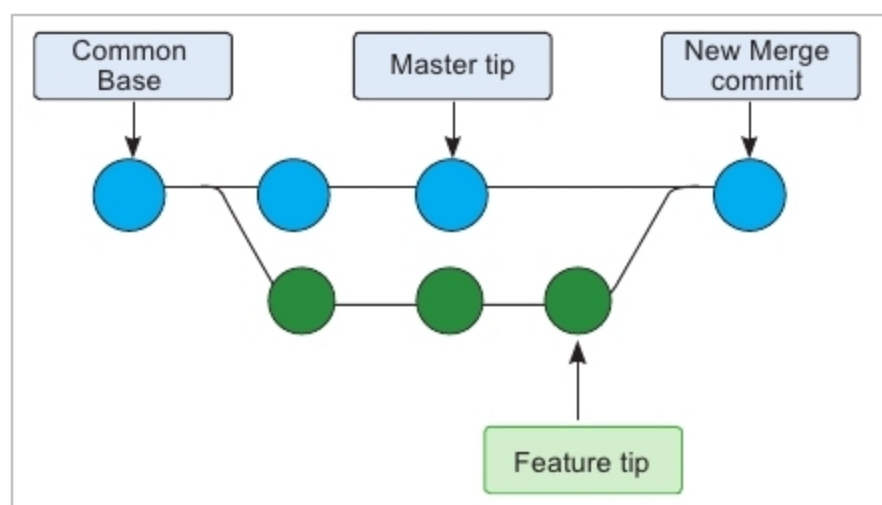


Figure 5: Snapshots of commits after merge. A new merge commit is created

Integrating the branches

There are two ways to put the work done in branches onto the main line of development — merge and rebase. Both ways take the series of commits from the target and current branch, and combine them into one unified history, but both do it in different ways.

Based on which branch needs to be updated from which one, the branches can be designated as source branch and target branch. In this scenario (refer Figure 3) we consider master as source branch and feature as target, assuming that work being done on the feature needs to be updated with work on the master.

Merge: Merging is done after checking out the target branch and then using `git merge <source-branch-name>`, which is shown in Figure 4.

`git merge` introduces a *merge commit* in the history of the feature branch, that binds the histories of both master and feature, and updates the feature with changes made to the master branch. Figure 3 shows the branching structure before *merge commit*. The branching structure after *merge commit* is shown in Figure 5.

Rebase: This has many more capabilities like rewriting the history interactively, re-sequencing the history, editing

```
(base) ekta@git-demo$ git branch
* feature
  master
(base) ekta@git-demo$ echo "Demonstrate rebasing" >> branch_file.txt
(base) ekta@git-demo$ git add branch_file.txt
(base) ekta@git-demo$ git commit -m "2nd commit on branch"
[feature cc3b2f8] 2nd commit on branch
1 file changed, 1 insertion(+)
```

Figure 6: Changing some files in the feature branch to demonstrate rebasing

```
(base) ekta@git-demo$ git checkout master
Switched to branch 'master'
(base) ekta@git-demo$ echo "new file again" > file_2.txt
(base) ekta@git-demo$ git add file_2.txt
(base) ekta@git-demo$ git commit -m "this commit will be rebased in feature branch"
[master 19b3902] this commit will be rebased in feature branch
1 file changed, 1 insertion(+)
create mode 100644 file_2.txt
```

Figure 7: Changing some files in the master branch to demonstrate rebasing

```
(base) ekta@git-demo$ git checkout feature
Switched to branch 'feature'
(base) ekta@git-demo$ git rebase master
First, rewinding head to replay your work on top of it...
Applying: first commit on a branch
Applying: 2nd commit on branch
(base) ekta@git-demo$ git log
commit a6b9fcd24045618718ec1a47b89c65ccd18679c4 (HEAD -> feature)
Author: Ekta Nandwani <Ekta.Nandwani@iitb.org>
Date: Thu May 7 19:41:52 2020 +0530

    2nd commit on branch

commit 6494d102784ef8f49da78c3a294b8f4eac93fff8
Author: Ekta Nandwani <Ekta.Nandwani@iitb.org>
Date: Thu May 7 19:35:15 2020 +0530

    first commit on a branch

commit 19b390252c51f2ba521aa6fcd6979498e891f42d (master)
```

Figure 8: Rebasing master onto feature